

```

import polars as pl
import requests
import pandas as pd
from datetime import datetime
import os

class SlovakFinanceDataDownloader:
    def __init__(self, output_dir='slovakia_finance_data'):
        self.output_dir = output_dir
        os.makedirs(output_dir, exist_ok=True)

    def download_ministry_finance_data(self):
        """
        Download datasets from Ministry of Finance Open Data Portal
        Note: Replace URLs with actual dataset URLs after manual verification
        """
        datasets = {
            'budget_revenue': 'https://opendata.mfsr.sk/api/v1/datasets/budget_revenue.csv',
            'budget_expenditure':
'https://opendata.mfsr.sk/api/v1/datasets/budget_expenditure.csv',
            'state_treasury': 'https://opendata.mfsr.sk/api/v1/datasets/state_treasury.csv'
        }

        for name, url in datasets.items():
            try:
                response = requests.get(url)
                response.raise_for_status()

                # Save CSV file
                filepath = os.path.join(self.output_dir,
f'{name}_{datetime.now().strftime("%Y%m%d")}.csv')
                with open(filepath, 'wb') as f:
                    f.write(response.content)

                # Convert to Polars DataFrame for quick processing
                df = pl.read_csv(filepath)
                print(f"Downloaded and processed {name} dataset")

                # Optional: Preview first few rows
                print(df.head())

            except Exception as e:
                print(f"Error downloading {name} dataset: {e}")

    def download_contracts_data(self):
        """
        Download data from Central Register of Contracts (CRZ)
        Note: This might require authentication or specific API calls

```

```

"""
# Example URL - replace with actual CRZ API endpoint
crz_url = 'https://www.crz.gov.sk/api/v1/contracts'

try:
    response = requests.get(crz_url)
    response.raise_for_status()

    filepath = os.path.join(self.output_dir,
f'government_contracts_{datetime.now().strftime("%Y%m%d")}.json')

    with open(filepath, 'wb') as f:
        f.write(response.content)

    print("Contracts data downloaded successfully")

except Exception as e:
    print(f"Error downloading contracts: {e}")

def merge_and_analyze(self):
    """
    Merge and perform initial analysis on downloaded datasets
    """
    # Read CSV files
    budget_revenue = pl.read_csv(os.path.join(self.output_dir, [f for f in
os.listdir(self.output_dir) if 'budget_revenue' in f][0]))
    budget_expenditure = pl.read_csv(os.path.join(self.output_dir, [f for f in
os.listdir(self.output_dir) if 'budget_expenditure' in f][0]))

    # Basic analysis
    total_revenue = budget_revenue['amount'].sum()
    total_expenditure = budget_expenditure['amount'].sum()

    print(f"Total Government Revenue: €{total_revenue:,.2f}")
    print(f"Total Government Expenditure: €{total_expenditure:,.2f}")
    print(f"Budget Balance: €{total_revenue - total_expenditure:,.2f}")

def main():
    downloader = SlovakFinanceDataDownloader()
    downloader.download_ministry_finance_data()
    downloader.download_contracts_data()
    downloader.merge_and_analyze()

if __name__ == '__main__':
    main()

```